

LAB 9 – Working With Local JSON Storage (Flutter + File Persistence)

1. Objective

In this lab, students will build three small Flutter tasks that teach how to store, read, and update data locally using JSON files.

This lab reinforces key concepts from **Module 9 – JSON & Local Storage**, including:

- Reading JSON from assets
- Writing JSON files to app storage
- Maintaining local persistence between app restarts
- Implementing simple CRUD operations on JSON data
- Designing UI that reflects local data changes

At the end, students will have a fully functional set of JSON-powered screens that run correctly in:

- DartPad (where applicable)
 - Android Studio
 - VS Code
-

2. Requirements

This lab is divided into **three sub-tasks**, each focusing on one major skill.

Lab 9.1 – Read JSON From Assets (Local Embedded File)

Lab 9.2 – Save & Load JSON From Device Storage

Lab 9.3 – JSON CRUD Mini Database (Search + Edit + Delete)

Students may complete them in **separate Flutter projects** OR in one combined project (recommended).

2.1 Functional Requirements

Lab 9.1 – Read JSON From Assets

The app must:

- Include a JSON file in the project /assets/
 - Load JSON using `rootBundle.loadString()`
 - Parse JSON into a Dart List
 - Display data using `ListView`
 - Show at least 2 fields per item
-

Lab 9.2 – Save & Load JSON Using Local Storage

The app must:

- Read JSON file from application documents directory
 - Save updates back to the JSON file
 - Persist data across app restarts
 - Provide minimal UI:
 - Add item
 - Show list
 - Save button (or auto-save)
-

Lab 9.3 – Local JSON CRUD + Search

The app must include:

- A JSON file used as a local mini-database
- Ability to:
 - Add item
 - Edit item
 - Delete item
 - Search/filter items
- Automatically save JSON after every change

- Clean and functional list UI
-

2.2 Technical Requirements

Your projects must:

- Use Flutter + Dart only
- Use dart:convert for JSON encoding/decoding
- Use path_provider for accessing local storage (Lab 9.2 & 9.3)
- Use setState or State Management from module 7 (optional)
- Run without errors on:
 - DartPad (Lab 9.1 and parts of Lab 9.3)
 - Android Studio
 - VS Code

Required Widgets & Methods:

- ListView / ListView.builder
 - FloatingActionButton
 - TextField
 - jsonDecode / jsonEncode
 - File read/write (Lab 9.2 & 9.3)
 - StatefulWidget
-

2.3 Optional Enhancements (Bonus)

Students may optionally:

- Add toast/snackbar messages after saving
- Add pull-to-refresh
- Add multiple JSON files

- Add confirm dialog before delete
- Auto-generate incremental IDs
- Create custom UI components

These are optional but improve your score.

3. Guided Steps

Lab 9.1 – Read JSON From Assets

Step 1 – Project Setup

Create new project and add JSON file inside assets/data/.

Step 2 – Register Assets in pubspec.yaml

Enable JSON file under flutter → assets.

Step 3 – Load JSON File

Use async loading (rootBundle.loadString) to read file content.

Step 4 – Decode JSON

Convert loaded string into a Dart List using JSON decode.

Step 5 – Display Data

Use ListView to show item fields.

Lab 9.2 – Save & Load JSON From Device Storage

Step 1 – Add path_provider

Add required dependency to pubspec.yaml.

Step 2 – Create a Storage Service

Implement helper methods to:

- Locate app document directory
- Read existing JSON file

- Create a new file if not found
- Write updated JSON back to storage

Step 3 – Load Data at Startup

Read file during initState and load items into a list.

Step 4 – UI for Adding Items

TextField + Add button → append items to list.

Step 5 – Save Button

Write list to JSON file and show confirmation message.

Lab 9.3 – JSON CRUD Mini Database

Step 1 – Local Data Structure

Prepare an in-memory list (loaded from JSON file).

Step 2 – Search Bar

Filter list by name or keyword.

Step 3 – Add / Edit / Delete

- Add: Create new item with unique ID
- Edit: Update fields of selected item
- Delete: Remove item after confirmation

Step 4 – Auto-Save

Re-save JSON file after each CRUD action.

Step 5 – Improved UI

Use ListTile or custom widgets to display item details.

4. Expected Result

Your final Lab 9 output should:

- Correctly load JSON from assets (Lab 9.1)
- Correctly save JSON to device storage and reload after restart (Lab 9.2)
- Provide full CRUD interaction with JSON "database" (Lab 9.3)
- Feature responsive UI that updates on changes
- Run on:
 - Android Studio
 - VS Code
 - DartPad (Lab 9.1 & partial Lab 9.3)

5. Submission

Students must submit:

Required Deliverables

- A ZIP file of project(s)
- Or DartPad links for applicable tasks
- JSON files used in each lab
- Screenshots:
 - JSON loaded into list (Lab 9.1)
 - Saving & reloading data (Lab 9.2)
 - CRUD + Search UI (Lab 9.3)

Evaluation Criteria

Category	Weight	Description
JSON Reading	25%	Correctly loads & parses assets JSON
JSON Writing	25%	Proper save/load functionality
Local Persistence	20%	Data persists after restart

Category	Weight	Description
CRUD Functionality	20%	Add, edit, delete, search
UI/UX Quality	10%	Clear layout, smooth interaction